

 **GAME_BEFORE.py**
Code **README(202610...**
Document

I have uploaded the project's `game.py` and `README.md` analyze the code and README and debug from step 1 step by step

Absolutely. We'll do this **step by step**, following the README's intended lab workflow rather than changing random things.

I've analyzed both `game.py` and the README. The README explicitly says there is **one deliberate bug + three optional features**, so we should tackle them in exactly that order.  README

Step 1 — Fix the deliberate bug

The README tells us exactly where the bug is: `trap_enemies()` .

Currently, your code has:

 Python

```
if bubble.pos.distance_squared_to(enemy.center) < 24 * 2:
```

The important issue is that:

 Python

```
distance_squared_to()
```

returns a **squared distance**.

But you're comparing it against:

 Python

```
24 * 2
```

which is only `48` , rather than the squared threshold $24^2 = 576$.

The README specifically identifies this mismatch.  README

What we should change

Change **only this line**:

 Python

```
if bubble.pos.distance_squared_to(enemy.center) < 24 * 2:
```

to:

</> Python

```
if bubble.pos.distance_squared_to(enemy.center) < 24 * 24:
```

So the complete function becomes:

</> Python

```
def trap_enemies(self, bubble):
    if bubble.enemy or bubble.age >= 1.5:
        return
    for enemy in self.enemies:
        if bubble.pos.distance_squared_to(enemy.center) < 24 * 24:
            self.enemies.remove(enemy)
            bubble.enemy, bubble.vel.x = enemy, bubble.vel.x * 0.2
    return
```

Why 24 * 24 ?

Think of it like this:

</> Plain text

Actual distance:
 24 pixels
 ↓
 $\text{distance}^2 = 24 \times 24$
 $= 576$

So:

</> Python

```
distance_squared_to(...)
```

→ gives something like 500 , 400 , 100 , etc.

Therefore the comparison needs to be:

```
</> Python  
distance_squared < 576
```

rather than:

```
</> Python  
distance_squared < 48
```

With the old code, the actual distance had to be less than approximately:

```
</> Plain text  
 $\sqrt{48} \approx 6.9$  pixels
```

That's why the bubble had to almost perfectly overlap the enemy.

With the fix, the bubble can come within **24 pixels** of the enemy, which matches the intended behavior.

Before we move to Step 2

Don't implement anything else yet.

For the lab's required iterative debugging process, do this first:

1. Open your `game.py` .
2. Find line ~153.
3. Change:

```
</> Python  
24 * 2
```

→

```
</> Python  
24 * 24
```

4. Save the file.
5. Run the game:

</> Bash

```
python game.py
```

6. Test whether bubbles now trap enemies when they **brush against them**, rather than requiring pixel-perfect overlap.

The rest of the trapping logic is already consistent with the README: once trapped, the enemy is removed from `self.enemies` and stored in `bubble.enemy` . [</> GAME_BEFORE](#)

Tell me what happens when you test Step 1 — whether it works, doesn't work, or you get an error. Then we'll move to **Step 2: `bubble_tint(bubble)`** .

task 1 worked succesfully now move on to task 2

Yesss, Task 1 is done 🎉 Let's move to **Task 2: `bubble_tint(bubble)`** .

The README says this function should return an `(r, g, b)` color, and specifically suggests **changing the bubble color based on how close it is to expiring**. [📖 README](#)

Step 2.1 — Understand the current function

Right now you have:

</> Python

```
def bubble_tint(bubble):
    """Return an (r, g, b) colour for a bubble, or None for the default."""
    pass
```

So currently it does **nothing**.

But `draw()` already calls it here:

</> Python

```
color = bubble_tint(bubble) or ((120, 230, 255) if bubble.enemy is None else (255, 190, 230))
pygame.draw.circle(screen, color, bubble.pos, bubble.radius, 2)
```

That means:

- If `bubble_tint()` returns a color → **that color is used**
- If it returns `None` → the existing default color is used

The defaults are:

Plain text

Normal bubble → (120, 230, 255)

Trapped enemy bubble → (255, 190, 230)

This is already implemented in your `draw()` function. `GAME_BEFORE`

Step 2.2 — What information does a Bubble have?

Look at your `Bubble` class:

Python

```
class Bubble:
    def __init__(self, x, y, direction):
        self.pos = pygame.Vector2(x, y)
        self.vel = pygame.Vector2(direction * 260, 0)
        self.age, self.enemy, self.life = 0.0, None, BUBBLE_LIFE
```

So every bubble has:

Python

```
bubble.age
bubble.enemy
bubble.life
```

And:

Python

```
BUBBLE_LIFE = 8.0
```

`GAME_BEFORE`

For Task 2, the README specifically suggests using:

```
bubble.life
```

to determine the color. [README](#)

Step 2.3 — Decide how we want the color to behave

A nice simple implementation is:

</> Plain text

Lots of life remaining

↓

cyan/blue

Getting close to expiry

↓

yellow

Very close to expiry

↓

red

This gives the player a visual warning that the bubble is about to disappear.

We don't need to modify the `Bubble` class or `draw()`.

We only implement the empty function.

Step 2.4 — Implement it

Replace:

</> Python

```
def bubble_tint(bubble):  
    """Return an (r, g, b) colour for a bubble, or None for the default."""  
    pass
```

with:

</> Python

```
def bubble_tint(bubble):  
    """Return an (r, g, b) colour for a bubble, or None for the default."""  
    if bubble.enemy is not None:  
        if bubble.life < 2:  
            return (255, 80, 80)  
        return (255, 190, 230)  
  
    if bubble.age > BUBBLE_TRAVEL:  
        return (120, 230, 255)
```

```
return None
```

But wait — there's an important detail

Your README says:

"Idea: color bubbles by how close they are to expiring (`bubble.life`)."  README

So let's make the implementation actually use `life`, rather than just checking whether the bubble has an enemy.

I'd recommend this cleaner version:

`</>` Python

```
def bubble_tint(bubble):
    """Return an (r, g, b) colour for a bubble, or None for the default."""
    if bubble.life < 2:
        return (255, 80, 80)
    elif bubble.life < 4:
        return (255, 200, 80)
    return None
```

What this does

Remaining life	Color
> 4 sec	Default
2–4 sec	Yellow
< 2 sec	Red

And because `draw()` already has the fallback:

`</>` Python

```
bubble_tint(bubble) or default_color
```

returning `None` for healthy bubbles preserves the existing appearance.

Step 2.5 — Test it

Put this version into your code:

</> Python

```
def bubble_tint(bubble):
    """Return an (r, g, b) colour for a bubble, or None for the default."""
    if bubble.life < 2:
        return (255, 80, 80)
    elif bubble.life < 4:
        return (255, 200, 80)
    return None
```

Then run:

</> Bash

```
python game.py
```

What you should observe

Blow a bubble and watch it.

Initially:

💙 **Normal cyan bubble**

As it gets closer to expiring:

🟡 **Yellow**

Then:

🔴 **Red**

One thing to notice: your `life` value only decreases while the bubble has an enemy trapped inside it:

</> Python

```
if self.enemy:
    self.life -= dt
```

So **only trapped bubbles will eventually change color**. Normal bubbles don't count down `life` ; they disappear by moving off-screen instead. </> `GAME_BEFORE`

That's actually consistent with the game's trap-and-release mechanic described in the README.

 README

Do this and test it. Tell me what you see. If Task 2 works, we'll move to Task 3 — `on_fruit_collected(fruit)` , where we'll add an actual gameplay effect rather than just changing a color.

explain this code please

Of course! Let's break down **Task 2** very slowly, because the main idea is actually quite simple once you understand what `bubble.life` and `None` are doing.

The code we're discussing is:

 Python

```
def bubble_tint(bubble):  
    """Return an (r, g, b) colour for a bubble, or None for the default."""  
    if bubble.life < 2:  
        return (255, 80, 80)  
    elif bubble.life < 4:  
        return (255, 200, 80)  
    return None
```

1. What is `bubble_tint()` ?

This is a function that answers one question:

"What color should this bubble be right now?"

It receives one argument:

 Python

```
bubble
```

which is a `Bubble` object from your game.

Your `Bubble` class has:

 Python

```
self.age, self.enemy, self.life = 0.0, None, BUBBLE_LIFE
```

and:

```
</> Python
```

```
BUBBLE_LIFE = 8.0
```

So a trapped bubble starts with:

```
</> Plain text
```

```
life = 8 seconds
```

and its life decreases over time.

2. First line

```
</> Python
```

```
if bubble.life < 2:
```

This asks:

"Does this bubble have less than 2 seconds of life remaining?"

For example:

```
</> Plain text
```

```
bubble.life = 1.5
```

Then:

```
</> Plain text
```

```
1.5 < 2
```

is **True**.

So Python executes:

```
</> Python
```

```
return (255, 80, 80)
```

3. What is (255, 80, 80) ?

That's an RGB color.

RGB means:

```
</> Plain text
```

(Red, Green, Blue)

So:

```
</> Python
```

```
(255, 80, 80)
```

means:

```
</> Plain text
```

```
Red    = 255
```

```
Green  = 80
```

```
Blue   = 80
```

Therefore it gives us a **red-ish color**.

So:

```
</> Python
```

```
return (255, 80, 80)
```

basically means:

"The bubble is about to expire, so make it red."

4. What does `elif` do?

Next we have:

```
</> Python
```

```
elif bubble.life < 4:
```

elif means:

"If the previous condition wasn't true, check this condition."

So suppose:

</> Plain text

```
bubble.life = 3
```

The first condition:

</> Python

```
bubble.life < 2
```

becomes:

</> Plain text

```
3 < 2
```

False.

So Python moves to:

</> Python

```
elif bubble.life < 4:
```

which becomes:

</> Plain text

```
3 < 4
```

True.

Therefore:

</> Python

```
return (255, 200, 80)
```

This is a yellow/orange color.

So:

</> Plain text

4+ seconds	→ normal/default color
2–4 seconds	→ yellow
<2 seconds	→ red

5. What happens when `life >= 4`?

This is where:

</> Python

```
return None
```

comes in.

Suppose:

</> Plain text

```
bubble.life = 7
```

First:

</> Python

```
7 < 2
```

False.

Then:

</> Python

```
7 < 4
```

False.

So Python reaches:

```
</> Python  
  
return None
```

This means:

"I don't want to specify a special color. Use the game's normal bubble color."

And your existing `draw()` code handles that:

```
</> Python  
  
color = bubble_tint(bubble) or ((120, 230, 255) if bubble.enemy is None else (255, 190, 230))
```

This is important.

6. What does `or` mean here?

This part can look scary:

```
</> Python  
  
bubble_tint(bubble) or default_color
```

But it's actually straightforward.

Case 1 — Function returns a color

Suppose:

```
</> Python  
  
bubble_tint(bubble)
```

returns:

```
</> Python  
  
(255, 80, 80)
```

Then:

```
</> Python
```

```
(255, 80, 80) or default_color
```

Python uses the color:

```
</> Plain text
```

```
(255, 80, 80)
```

So the bubble becomes red.

Case 2 — Function returns None

Suppose:

```
</> Python
```

```
bubble_tint(bubble)
```

returns:

```
</> Python
```

```
None
```

Then Python uses the thing after or :

```
</> Python
```

```
(120, 230, 255)
```

or:

```
</> Python
```

```
(255, 190, 230)
```

depending on whether the bubble contains an enemy.

So None basically means:

"Don't override the game's existing color."

7. Let's visualize the whole process

Imagine a trapped bubble starts with:

</> Plain text

```
life = 8
```

Then:

</> Plain text

```
8 seconds
```

↓

```
bubble.life = 8
```

↓

```
8 < 2? NO
```

```
8 < 4? NO
```

↓

```
return None
```

↓

```
use normal pink bubble
```

Then later:

</> Plain text

```
3 seconds
```

↓

```
bubble.life = 3
```

↓

```
3 < 2? NO
```

```
3 < 4? YES
```

↓

```
return (255, 200, 80)
```

↓

```
YELLOW bubble
```

Then:

</> Plain text

```
1 second
  ↓
bubble.life = 1
  ↓
1 < 2? YES
  ↓
return (255, 80, 80)
  ↓
RED bubble
```

So the player gets a visual warning:

</> Plain text

NORMAL →  → 

8. One very important thing from YOUR code

Your `Bubble.update()` contains:

</> Python

```
if self.enemy:
    self.life -= dt
```

So `life` only decreases when:

</> Python

```
bubble.enemy
```

is present.

That means this color-changing system is really telling the player:

"Your trapped enemy bubble is getting close to escaping."

And when:

</> Python

```
bubble.life <= 0
```

your game does:

</> Python

```
self.release(bubble)
```

So the overall sequence is:

</> Plain text

```
Enemy gets trapped
    ↓
bubble.enemy = enemy
    ↓
life starts decreasing
    ↓
8 sec → 4 sec → 2 sec → 0
    ↓
normal → yellow → red
    ↓
life reaches 0
    ↓
enemy escapes
```

That's exactly why `bubble.life` is a useful thing to visualize for this task. Your README specifically suggests using `bubble.life` for the tint. [📖 README](#)

The key idea to remember

If you need to explain this in your lab/viva:

`bubble_tint()` checks the remaining lifetime of a trapped bubble. If less than 4 seconds remain, it changes the bubble to yellow, and if less than 2 seconds remain, it changes it to red. Otherwise it returns `None`, allowing the existing default bubble color to be used.

And the order matters: **`< 2` has to come before `< 4`**, because a bubble with 1 second left also satisfies `1 < 4`.

ok this works and i have pushed the changes now task 3

Yesss, Task 2 is officially done ✅ and pushed. Let's do **Task 3: `on_fruit_collected(fruit)`**.

The README describes this as an optional feature: the function is called **immediately after the player collects a fruit**, and suggests either a sparkle effect or a running fruit counter in the HUD.

[📖 README](#)

Step 3.1 — Understand what is already happening

Right now the function is empty:

</> Python

```
def on_fruit_collected(fruit):  
    """Called when the player picks up a fruit; add a sound, sparkle, or bonus effect here."""  
    pass
```

But `update()` already handles the actual collection:

</> Python

```
if fruit.rect.colliderect(player.rect):  
    self.score += fruit.value  
    self.fruits.remove(fruit)  
    on_fruit_collected(fruit)
```

So the order is:

</> Plain text

```
Player touches fruit  
    ↓  
Add fruit's points to score  
    ↓  
Remove fruit from game  
    ↓  
Call on_fruit_collected(fruit)
```

This means **Task 3 isn't about making fruit collection work** — that already works.

We're adding an extra effect when collection happens.

What should we implement?

Since `on_fruit_collected()` currently has no access to the `Game` object, the easiest useful feature is a **sound effect**.

But there is a problem: Pygame's mixer needs to be initialized and we'd need an actual sound file. Your project only contains `game.py` and `README.md`, so that would add unnecessary setup.

The README gives another option: a **sparkle effect**.  [README](#)

However, a proper sparkle animation would require storing particles somewhere, and this function doesn't have access to `Game.fruits` , `Game.bubbles` , etc.

So I recommend the simplest feature that fits the existing architecture:

Add a fruit collection counter.

But because `on_fruit_collected(fruit)` is a standalone function, we'd need somewhere to store that counter.

We can use a small module-level variable:

`</>` Python

```
fruits_collected = 0
```

and then:

`</>` Python

```
def on_fruit_collected(fruit):  
    global fruits_collected  
    fruits_collected += 1
```

However, I'd actually avoid this approach because global state isn't ideal in an object-oriented game.

A cleaner implementation for *this particular lab* is to use the function to modify the fruit itself, but that won't visibly do anything because the fruit is immediately removed.

So before changing the architecture, let's look at what the README actually requires: it says the return value is ignored and gives **sparkle effect or running collected-fruit counter** only as ideas.

 README

We therefore have freedom to choose the feature.

I recommend: bonus effect based on fruit value

We can make collection trigger a small **console message**, which requires no additional assets:

`</>` Python

```
def on_fruit_collected(fruit):  
    print(f"Fruit collected! +{fruit.value} points")
```

This is technically a working feature, but honestly, it's not very interesting for a game.

Better: let's add a proper on-screen notification, but that requires a small structural change to `Game` .

So let's do Task 3 properly rather than taking a shortcut.

One question before we code

Do you want Task 3 to be:

- A. 🍓 **Fruit counter in HUD** — e.g. `Fruits 3` , simple and clean
- B. ✨ **Sparkle/particle effect** when fruit is collected — more visually impressive
- C. 🎧 **Collection sound** — requires adding a sound asset

I'd recommend **B** for making the project look more polished, but **A** is the easiest and safest for your lab.

only a is enough

Yep — **A is more than enough** and it's clean for the lab. 👍

We'll add a **fruit counter to the HUD**. The important part is that we should do it with the existing `Game` structure rather than introducing unnecessary global variables.

Step 3.1 — Add the counter to `Game.reset()`

Find:

</> Python

```
def reset(self):
    self.level, self.score, self.lives, self.combo, self.state = 1, 0, 3, 0, "play"
    self.bonus_awarded = 0
```

Add:

</> Python

```
self.fruits_collected = 0
```

So it becomes:

</> Python

```
def reset(self):
    self.level, self.score, self.lives, self.combo, self.state = 1, 0, 3, 0, "play"
    self.bonus_awarded = 0
    self.fruits_collected = 0
```

```
self.player = Player()
self.fruits = []
self.start_level()
```

This means every time you reset the game, the fruit counter starts at `0` .

Step 3.2 — Update the counter when a fruit is collected

Currently you have:

</> Python

```
if fruit.rect.colliderect(player.rect):
    self.score += fruit.value
    self.fruits.remove(fruit)
    on_fruit_collected(fruit)
```

Change it to:

</> Python

```
if fruit.rect.colliderect(player.rect):
    self.score += fruit.value
    self.fruits.remove(fruit)
    self.fruits_collected += 1
    on_fruit_collected(fruit)
```

So now:

</> Plain text

```
Player touches fruit
    ↓
Score increases
    ↓
Fruit disappears
    ↓
fruits_collected increases by 1
    ↓
on_fruit_collected() is called
```

The existing collection logic is in your `update()` function. </> GAME_BEFORE

Step 3.3 — What do we do with `on_fruit_collected()` ?

Since we've chosen the counter approach, we **don't actually need this function to perform the counting**.

The README explicitly says its return value is ignored and suggests a running collected-fruit counter as one possible feature.  README

We can therefore leave:

 Python

```
def on_fruit_collected(fruit):  
    """Called when the player picks up a fruit; add a sound, sparkle, or bonus effect here."""  
    pass
```

as-is.

But then the function isn't really being "implemented," which isn't ideal for the task.

So let's make it do something harmless and useful: add a comment explaining that the counter is handled by `Game.update()` .

Actually, **don't do that**—comments aren't implementation.

Better approach

Let's make the function return the fruit's value:

 Python

```
def on_fruit_collected(fruit):  
    """Called when the player picks up a fruit; add a sound, sparkle, or bonus effect here."""  
    return fruit.value
```

But since the return value is ignored, this doesn't visibly contribute anything either.

So I recommend we **move the counter logic into the function properly**, using the game's state. That would require changing the function signature to receive `game` , which conflicts with the README's specified `on_fruit_collected(fruit)` interface.

Therefore, for this lab, the cleanest interpretation is:

- `on_fruit_collected(fruit)` remains the extension hook.
- `Game.fruits_collected` handles the actual counter.
- The HUD displays the counter.

That still fulfills the "running collected-fruit counter" feature suggested by the README.

Step 3.4 — Display it in the HUD

Find:

`</>` Python

```
hud = self.font.render(f"Score {self.score}  Lives {self.lives}  Level {self.level}  R = reset",
```

Change it to:

`</>` Python

```
hud = self.font.render(  
    f"Score {self.score}  Lives {self.lives}  Level {self.level}  Fruits {self.fruits_collected}  
    True,  
    (240, 240, 240)  
)
```

Your HUD will now look something like:

`</>` Plain text

```
Score 500  Lives 3  Level 1  Fruits 2  R = reset
```

Step 3.5 — Test it

Run:

`</>` Bash

```
python game.py
```

Then:

1. Trap an enemy.
2. Pop the bubble.
3. A fruit should appear.
4. Touch the fruit.

5. Check the HUD.

You should see:

```
</> Plain text
```

```
Fruits 1
```

Collect another:

```
</> Plain text
```

```
Fruits 2
```

And so on.

Press `R` and it should reset to:

```
</> Plain text
```

```
Fruits 0
```

because `reset()` sets:

```
</> Python
```

```
self.fruits_collected = 0
```

So your three changes are:

1. `reset()`

```
</> Python
```

```
self.fruits_collected = 0
```

2. Fruit collection

```
</> Python
```

```
self.fruits_collected += 1
```

3. HUD

```
</> Python
```

```
Fruits {self.fruits_collected}
```

That's all you need. Don't change anything else yet.

Test this and **tell me whether the counter works**. Then we'll move to **Task 4**:

bonus_life_threshold() , which is actually the easiest one.

done it works

YAY 🎉 Task 3 is done.

You now have:

-  **Task 1:** Fixed bubble trapping distance calculation.
-  **Task 2:** Bubble color changes as the trapped bubble approaches expiration.
-  **Task 3:** Added a fruit counter to the HUD.

Let's finish the final task.

Task 4 — **bonus_life_threshold()**

The README tells us exactly what this function is supposed to do:

Return an integer score threshold. Whenever the player's score crosses a multiple of that threshold, they receive one extra life. The bookkeeping is already implemented.  [README](#)

Right now your function is:

```
</> Python
```

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus liv  
    pass
```

So currently it returns nothing.

Step 4.1 — Look at the existing code

You already have this in `update()` :

```
</> Python
```

```
threshold = bonus_life_threshold()
```

```
if threshold and self.score // threshold > self.bonus_awarded:  
    self.bonus_awarded = self.score // threshold  
    self.lives += 1
```

We don't need to change this.

The game has already implemented the bonus-life mechanism for us.

We only need to tell it:

"Give the player an extra life every 5000 points."

Step 4.2 — Implement the function

Replace:

</> Python

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus liv  
    pass
```

with:

</> Python

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus liv  
    return 5000
```

That's literally it. 🤔

The README itself suggests `5000` as the intended idea.  [README](#)

Step 4.3 — But understand why it works

Suppose:

</> Plain text

```
threshold = 5000
```

The existing code calculates:

```
</> Python
```

```
self.score // threshold
```

// is integer division.

So:

Score	score // 5000
0	0
1,000	0
4,999	0
5,000	1
6,000	1
9,999	1
10,000	2
15,000	3

And `self.bonus_awarded` keeps track of how many bonus lives have already been given.

Initially:

```
</> Python
```

```
self.bonus_awarded = 0
```

At 5,000 points:

```
</> Plain text
```

```
score // threshold  
= 5000 // 5000
```

```
= 1
```

Then:

```
</> Python
```

```
1 > 0
```

True.

So:

```
</> Python
```

```
self.bonus_awarded = 1  
self.lives += 1
```



Extra life!

What happens at 6,000?

```
</> Plain text
```

```
6000 // 5000 = 1
```

But now:

```
</> Plain text
```

```
bonus_awarded = 1
```

So:

```
</> Plain text
```

```
1 > 1
```

False.

No second life.

That's important because otherwise you'd gain a life every frame after reaching 5,000 points.

At 10,000 points

</> Plain text

```
10000 // 5000 = 2
```

Now:

</> Plain text

```
2 > 1
```

True.

So you get another life:

</> Plain text

```
bonus_awarded = 2  
lives += 1
```

Therefore the behavior is:

</> Plain text

```
0 points
```

↓

```
3 lives
```

```
5,000 points
```

↓

```
4 lives
```

```
10,000 points
```

↓

```
5 lives
```

```
15,000 points
```

↓

```
6 lives
```

That's exactly what the existing bookkeeping was designed to accomplish.

Step 4.4 — Test it

Change the function to:

</> Python

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus liv  
    return 5000
```

Then save and run:

</> Bash

```
python game.py
```

You can test normally, but getting to 5,000 points could take a while.

For testing **only**, you can temporarily change:

</> Python

```
return 5000
```

to:

</> Python

```
return 500
```

Then get 500 points and check that your lives increase.

Once you've confirmed it works, **change it back to 5000** before committing.

After Task 4

You'll have completed **all four README tasks**:

Task	Status
1. Fix bubble trapping	

Task	Status
2. Bubble tint	✓
3. Fruit counter	✓
4. Bonus life	🔧 Now

After you test Task 4, **don't push immediately**. Tell me whether it works first. Then we'll do a **final code review of the entire `game.py` against the README**, make sure we haven't introduced anything unnecessary, and I'll help you prepare exactly what you need for the **before/after 10-second videos and submission checklist**. [📖 README](#)

so this part of the code will need not be seen right now right only when we reach 5000 points

Exactly 👍

You **don't need to do anything visibly with that code until the score reaches 5,000**.

But there's an important distinction:

Python

```
def bonus_life_threshold():  
    return 5000
```

This function is actually **called every frame** by:

Python

```
threshold = bonus_life_threshold()
```

So the function itself is running all the time. It's just that **nothing happens until the score reaches a multiple of 5,000**.

For example:

Plain text

```
Score = 3000  
3000 // 5000 = 0  
→ no extra life  
  
Score = 4999
```

```
4999 // 5000 = 0
```

```
→ no extra life
```

```
Score = 5000
```

```
5000 // 5000 = 1
```

```
→ +1 life 🎉
```

```
Score = 6000
```

```
6000 // 5000 = 1
```

```
→ nothing
```

```
Score = 10000
```

```
10000 // 5000 = 2
```

```
→ +1 more life 🎉
```

So yes, during normal gameplay you won't see any change until you hit 5,000 points. The HUD will continue showing your current lives, and when you cross 5,000, it should increase by 1.

For testing, you can temporarily use `return 500`, verify the mechanism, and then change it back to `return 5000` before your final push.

ok done i couldnt reach 5000 points but the code runs smoothly

Perfect — that's completely fine. ❤️ You don't need to prove the 5,000-point case manually if the logic is correct and the game runs smoothly.

At this point:

-  **Task 1:** Bubble trapping bug fixed
-  **Task 2:** Bubble tint implemented
-  **Task 3:** Fruit counter implemented
-  **Task 4:** Bonus-life threshold implemented
-  Game runs without errors

Your final `bonus_life_threshold()` is simply:

```
</> Python
```

```
def bonus_life_threshold():
```

```
    """Return a score value at which the player earns an extra life, or None to disable bonus liv
    return 5000
```

And the existing bookkeeping handles the actual life award. The README explicitly says that bookkeeping is already implemented and only asks you to choose the threshold.  [README](#)

One thing I'd do before we call the project finished

Since you've now completed all four tasks, **don't change anything else yet.**

Next, we should do a **final verification pass** against the README:

1. Check the complete `game.py`
2. Make sure all 4 changes are actually present
3. Make sure there are no accidental changes
4. Check that the game behavior matches the README's expected behavior
5. Check what you need to show in your **before/after 10-second videos**
6. Make sure your GitHub folder has the correct files
7. Then you can make your final commit/push

The README's submission requires the two gameplay videos plus the complete Chat/LLM page link/history.  README

If you want, **send me your final `game.py` after all your changes**, and we'll do that final verification step-by-step before you submit.

 **game(1).py**
Code

this is the final file

Yep — I checked the final `game.py`. The four task changes are present:

- **Task 1:** `24 * 24` squared-distance fix is present.  `game`
- **Task 2:** `bubble_tint()` is implemented with age-based colors.  `game`
- **Task 3:** `fruits_collected` is initialized, incremented, and displayed in the HUD.  `game`
 `game`
- **Task 4:** `bonus_life_threshold()` returns `5000`.  `game`

One thing I noticed

Your **Task 2** implementation is slightly different from the version we discussed earlier:

 Python

```
if bubble.age < 1:
    return (0, 0, 255)
elif bubble.age < 2:
    return (0, 255, 0)
elif bubble.age < 3:
    return (255, 255, 0)
else:
    return (255, 0, 0)
```

This uses **`bubble.age`** , not `bubble.life` . That's completely valid as an implementation of `bubble_tint()` , and your game is apparently working with it. The README only gives `bubble.life` as an *idea*, not a strict requirement. [↗ GAME_BEFORE](#)

So I would not change it now if you've already tested it and pushed it.

Final verdict

Your final file contains all four requested task modifications and the game structure remains intact. I don't see a reason to make another functional change before submission.

The only thing I'd recommend now is **don't touch the code further.** 😭

Next, we should handle the **submission evidence**: the README requires a 10-second **before** video showing the original bug, a 10-second **after** video showing the fixes/features, and the Chat/LLM page link with the complete chat history. [📄 README](#)

If you want, I can walk you through **exactly what to record in the before and after videos, what screenshots to take, and what to submit on GitHub**, step by step.

yes that i only changed so we can see the colours distinctly

Exactly! That makes sense. 😊 The distinct **blue → green → yellow → red** progression makes the feature much easier to demonstrate in your video and for whoever grades it to immediately see that Task 2 is working.

So **keep your current Task 2 implementation**. It is intentionally more visually obvious than the subtle expiration-based colors we initially discussed.

Your final code is good to go. ✅

Now let's move to the **submission evidence**, because the README specifically requires:

1. 🎥 **10-second video before changes** — showing the original bug/broken behavior
2. 🎥 **10-second video after changes** — showing the bug fixed + new features
3. 🔗 **Chat/LLM page link** with the complete chat history [📄 README](#)

For the BEFORE video

You need to demonstrate **Task 1's original bug**.

The best recording would be:

- Start the original `game.py` **before your Task 1 change**
- Blow a bubble toward an enemy
- Make the bubble visibly pass very close to the enemy without trapping it

- Keep the recording around 10 seconds

The point is to make the bug obvious: the original code used an inconsistent squared-distance threshold, so only an almost pixel-perfect overlap could trigger trapping.

For the AFTER video

You want to pack the completed features into ~10 seconds:

Suggested sequence:

</> Plain text

```
0-2 sec    → Blow bubble toward enemy
2-4 sec    → Bubble successfully traps enemy
4-6 sec    → Pop the trapped bubble → fruit appears
6-8 sec    → Collect fruit → HUD fruit counter increases
8-10 sec   → Show bubble color changes / gameplay
```

You don't necessarily have to demonstrate the 5,000-point bonus life in the video—the README asks for the bug fixed and **new features working**, and you weren't able to naturally reach 5,000. The other implemented features give you much more visible evidence.

And because you deliberately made the bubble colors very distinct, **Task 2 should be especially easy to show.** 🎨

If you're ready, I can give you the **exact recording procedure for the BEFORE video first**, including how to temporarily get your old buggy version back without messing up your final code.